



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Julia for Joint Models

Master Thesis

Yannik Ammann

August, 2023

Advisors: Prof. Peter Bühlmann

Supervisors at Roche: Dr. Daniel Sabanes Bove, Dr. Francois Mercier, Dr. Doug Kelkhoff

SfS, Department of Mathematics, ETH Zürich

Abstract

This work implements joint models for a longitudinal process influencing a time-to-event process in Julia. The goal was to develop a tool and workflow to allow researchers to implement their joint models and iterate and explore them. Another goal was to evaluate the Julia programming language and ecosystem concerning the development of statistical tools. I have developed workflows showing how to implement survival models, mixed-effects longitudinal models, and joint models in Julia. In addition, I have developed and described how to generate diagnostic statistics and plots for these models. These workflows show the simplicity and capabilities of the Julia language. One particular achievement is the close mapping of mathematical and statistical ideas to their implementation in code.

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Bayesian modeling and Turing.jl	2
2 Mathematical Formulations	5
2.1 Survival Analysis	5
2.2 Longitudinal process	8
2.3 Joint Models	10
3 Methods and Design	13
3.1 Towards implementation of Joint Models	13
3.2 Case Study: Simulated Data	15
3.3 Diagnostics	17
3.4 Comparison to Stan	21
4 Implementation of Joint Models	25
4.1 Type: HazardBasedDistribution	25
4.2 Usage	27
5 Conclusion	31
A Appendix	33
A.1 Further Exploration	33
A.2 Survival Workflow	33
A.3 Longitudinal Workflow	34
A.4 Joint Molder Workflow	34
A.5 Julia in Latex	34
Bibliography	35

Chapter 1

Introduction

This is an exploratory project to evaluate the Julia programming language and its ecosystem. Julia is a relatively new programming language released in 2017 [4] with the development starting in 2012.

To judge the development process in Julia and to test the tools available I implemented joint models. This is a complicated model with applications in clinical statistics. In the next sections I will give some background and motivation for the research question and give a glimpse into Bayesian modeling.

1.1 Motivation

Joint models describe the relationship between longitudinal measurements and time-to-event survival outcomes¹. In pharmaceutical research joint models can be used to assess the efficacy of novel treatments, optimize patient care strategies by individual prediction of outcomes or explore disease progression mechanisms. Joint models offer a valuable tool to gain insights from complex data by modeling the longitudinal process and patient survival through the joining of two separate models with an additional linking function.

analyzing longitudinal processes and survival times leading to more informed decision-making and improved patient outcomes [17].

The difficulty of joint models is the combination of measurements of a continuous process and survival outcomes. Available packages for joint models such as JM [19] and JMBayes [20] in the R programming language are not capable to describe non-linear longitudinal models. One achievement of this work is to implement the joint models described by Kerioui, Mercier, Bertrand et al. [17], which features a non-linear longitudinal mixed-effects model. Gower-Page, Kazantzidis, Sabanes Bove et al. started the development of an alternative

¹Chapter 2 explains these processes.

R-package called `jmpost` [11], which allows the usage of non-linear longitudinal models.

The R-package `jmpost` uses the C++ library Stan [22] and R bindings [23], which is an example of the so-called two-language problem [21]. One programming language is used for efficient algorithms and another for the researcher developing and iterating models. This makes the implementation inherently complex and developers and researchers have to be proficient in multiple programming languages, for example R and C++/Stan to push the boundaries of knowledge. This results in inefficiencies in development and debugging because of fragmented code in different languages. Additionally, flexibility to iterate and explore joint models is lacking.

To solve the two language problem and still be able to formulate non-linear joint models and sample efficiently, we want to explore the Julia programming language. Julia solves the two-language problem by being efficient with a just in time compiler, which turns Julia code into native machine code. In addition, Julia has the ecosystem for developing complex statistical tools such as joint models in an intuitive and flexible manner. The goal is to enable researchers to deploy and refine joint models with the highest efficiency and clarity to unravel the complicated relationships between survival and longitudinal observations.

1.2 Bayesian modeling and Turing.jl

Bayesian modeling combines the prior beliefs about parameters of a model with the available data. We update our beliefs based on the observed data resulting in a posterior distribution characterizing the uncertainty about the parameters. Using the posterior distribution, we can make predictions or analyze new data.

Bayes theorem states

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

For Bayesian inference we take A to represent the model parameters and B to represent the observations. Meaning A contains the parameters and formulation of a model that tries to describe the measurements B .

This makes $P(A)$ the prior beliefs of the parameters of a model and $P(A|B)$ we call the posterior distribution of the model parameters given the observations B . Probability $P(B|A)$ describes the likelihood of measurements B given model parameters A .

Finally, $P(B)$ is the probability of measurements. This is generally unknown. Thus Bayesian inference relies on the proportion

$$P(A|B) \propto P(B|A)P(A).$$

In chapter 2 I explicitly calculate the likelihood for joint models, since this is what we need to implement joint models with Bayesian inference.

In Julia Turing.jl [10] has established itself as a popular probabilistic programming framework for implementing Bayesian sampling algorithms. In Appendix A I provide workflows showcasing how to work with framework.

Chapter 2

Mathematical Formulations

In this chapter I present the formulation of joint models and calculate its log likelihood which is a key component in Bayesian sampling algorithms [10]. For this we first look at survival and longitudinal models separately before joining them via a link to build joint models.

2.1 Survival Analysis

In survival analysis we consider a random variable T which is non negative $T \geq 0$ and we are interested in its distribution function. The random variable T describes the time to an event which is a point of interest. In pharmaceutical survival models, these events could be an adverse reaction to a drug, death or other events of interest [6].

For such a non negative random variable T we define the corresponding cumulative distribution function (CDF) F

$$F : \mathbb{R}_{\geq 0} \rightarrow [0, 1] \quad F(t) \quad := P(T \leq t),$$

as well as probability density function (PDF) $f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ via the integral

$$\int_0^t f(u) du = F(t) = P(T \leq t).$$

In survival analysis we are especially interested in the complementary cumulative distribution function (CCDF) $F^c(t) := 1 - F(t) = P(T > t)$, which we will call survival function S

$$\begin{aligned} S : \mathbb{R}_{\geq 0} &\rightarrow [0, 1] \\ S(t) &:= P(T > t), \end{aligned}$$

2. MATHEMATICAL FORMULATIONS

since it represents the probability that no event occurred until time t i.e. survival until time t .

Furthermore, we formulate a hazard function $h : \text{supp}(S)^1 \subseteq \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ to be the conditional probability of an event occurring at time t with the condition that the event did not occur until time t

$$\begin{aligned}
 h(t) &:= \lim_{\delta \rightarrow 0^+} \frac{P(t \leq T \leq t + \delta | T > t)}{\delta} \\
 &= \lim_{\delta \rightarrow 0^+} \frac{1}{\delta} \left(\frac{P(t \leq T \leq t + \delta)}{P(T > t)} \right) \quad (\text{conditional probability}) \\
 &= \lim_{\delta \rightarrow 0^+} \frac{1}{\delta} \left(\frac{F(t + \delta) - F(t)}{S(t)} \right) \\
 &= \lim_{\delta \rightarrow 0^+} \underbrace{\left(\frac{F(t + \delta) - F(t)}{\delta} \right)}_{\text{derivative: } F'(t)=f(t)} \frac{1}{S(t)} \\
 &= \frac{f(t)}{S(t)}.
 \end{aligned}$$

The hazard function can be used to formulate the PDF and Survival function and thus describe a distribution. Let us try to derive the survival function S from the hazard. Notice that

$$f(t) = \frac{d}{dt}F(t) = \frac{d}{dt}(1 - S(t)) = -\frac{d}{dt}S(t),$$

thus

$$h(t) = \frac{f(t)}{S(t)} = \frac{-\frac{d}{dt}S(t)}{S(t)} = -\frac{d}{dt} \log S(t).$$

Formulating the cumulative hazard function $H(t) : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ and plugging in the hazard formulation above

$$H(t) := \int_0^t h(u) du = \int_0^t -\frac{d}{du} \log S(u) du = -\log S(t).$$

Solving for S we get

$$S(t) = \exp(-H(t)) = \exp\left(-\int_0^t h(u) du\right).$$

¹We need the support of S for the conditional probability. Note that the support of S is an interval since S is monotonically decreasing and $S(0) = 1$.

Finally we get a formulation of the probability density function f based on the hazard

$$h(t) = \frac{f(t)}{S(t)} = \frac{f(t)}{\exp\left(-\int_0^t h(u) du\right)}$$

$$f(t) = h(t) \exp\left(-\int_0^t h(u) du\right).$$

Later on in the section about joint models 2.3 we will alter the formulation of the hazard and thus of S and f based upon a longitudinal process.

Censoring and likelihood

When measuring processes in the real world, sometimes you do not measure an event and only record the absence of an event at a given time. Censoring describes how to handle these situations. Here we will focus on so-call "right"-censoring [6].

Suppose we have a time to event random variable T and a realization $\bar{t}$ as well as a censoring time t_c , we call an event right-censored if $t_c < \bar{t}$. When taking measurements we can record a time

$$t = \min\{t_c, \bar{t}\},$$

as well as an event indicator²

$$\delta = \begin{cases} 1 & \bar{t} < t_c \\ 0 & \bar{t} \geq t_c. \end{cases}$$

Leaving us with a measurement³ (t, δ) .

What censoring describes is how to calculate the likelihood for such a measurement. In right censoring we record the censored time as "survived". The likelihood of a particular hazard which defines the PDF f and the survival function S . The likelihood of the event (t, δ) given the distribution described by the hazard h is calculated as

$$L((t, \delta)|h) = f(t)^\delta \cdot S(t)^{1-\delta} = \begin{cases} f(t) & \bar{t} < t_c \quad (\text{event time is recorded}) \\ S(t) & \bar{t} \geq t_c \quad (\text{event time is not recorded}). \end{cases}$$

The argumentation for this case distinction is that for censored events $\delta = 0$ we know that no event occurred until time t . We can say that there is survival until time t , meaning the absence of an event. This is described by the CCDF $S(t)$.

²Denoting an event is **not** censored.

³This is specific to right-censoring, other methods can have different measurement structures

Covariates

To incorporate a covariates vector $x \in \mathbb{R}^l, l \in \mathbb{N}$ we can use the exponential function. This is called the Cox proportional-hazard model based on [7]

$$h(t|x) = h_0(t) \exp(\beta \cdot x)$$

Here $\beta \in \mathbb{R}^l$ are the covariate coefficients and h_0 is the baseline hazard that could be a parametric distribution⁴.

One reason for using an exponential is the possibility of comparing the proportionality of covariates x_1 and x_2 by

$$\begin{aligned} \frac{h(t|x_1)}{h(t|x_2)} &= \frac{h_0(t) \exp(\beta \cdot x_1)}{h_0(t) \exp(\beta \cdot x_2)} \\ &= \frac{\exp(\beta \cdot x_1)}{\exp(\beta \cdot x_2)} \\ &= \exp(\beta \cdot (x_1 - x_2)). \end{aligned}$$

The exponential in the last line contains the linear combination of the covariates which helps with their interpretation.

2.2 Longitudinal process

The longitudinal process describes measurements at different time points that are connected with each other. In clinical statistics these could belong to the same individual, measuring biomarkers such as tumor growth, test results or molecule concentration in blood.

Another complexity of the longitudinal process is that we are working with mixed effects i.e. hierarchical models. In the following we will refer to individuals to separate multiple longitudinal processes. This is specific to clinical statistics and could be abstracted into separate processes with the same underlying model structure.

Assume we have $N \in \mathbb{N}$ individuals and each individual $i \in [N]$ has $n_i \in \mathbb{N}$ measurements. For simplicity we consider only one such longitudinal process in a scalar space and do not consider covariates. The set of measurements we describe as $\{y_{ij}\}_{i \in [N], j \in [n_i]}$, where i denotes the individual and j the measurement number for this individual. For each measurement, we also have an associated time $\{t_{ij}\}_{i \in [N], j \in [n_i]}$ at which the measurement was taken.

⁴Exponential, Weibull, Log-Logistic, Log-Normal are examples [6].

A longitudinal model $m : T \times P \rightarrow \mathbb{R}$ describes a function of time $T := [0, \infty)$ and a parameter space $P := \mathbb{R}^l$ for some $l \in \mathbb{N}$. For every individual⁵ $i \in [N]$ we have a parameter ψ_i . We define $m_i : T \rightarrow \mathbb{R}, m_i(t) := m(t, \psi_i)$ as the individual longitudinal model for the individual i .

Notice that in this formulation, we do not assume linearity as is done in previous R implementations of joint models, namely JM [19] and JMBayes[20]. This general formulation allows non-linear mixed effects models such as in the paper by Kerouhi et al. [17] to be used.

Likelihood

To formulate a statistical model, we include an error process⁶. This gives us a probability density function $f_{\text{err}}(y_{ij})$ for any measurement y_{ij} . These error process is described by parameters which we include in the parametric description of the longitudinal model.

To calculate the likelihood, we consider a set $\mathcal{M} := \{m_i\}_{i \in [N]}$ of individual models. This includes the parameters for the longitudinal prediction model and for the measurement error. As well as the set of measurements with associated time points $Y_L = \{(t_{ij}, y_{ij})\}_{i \in [N], j \in [n_i]}$.

For a single measurement (y_{ij}, t_{ij}) we calculate the likelihood as

$$L((y_{ij}, t_{ij}) | \mathcal{M}) = L((y_{ij}, t_{ij}) | m_i) = f_{\text{err}}(y_{ij})(m_i(t_{ij})).$$

This extends to the log likelihood formulation for all measurements

$$\begin{aligned} l(\{y_{ij}\}_{i \in [N], j \in [n_i]} | \mathcal{M}) &= \log L(\{y_{ij}\}_{i \in [N], j \in [n_i]} | \mathcal{M}) \\ &= \log \left(\prod_{i \in [N]} \prod_{j \in [n_i]} L(y_{ij} | m_i) \right) \quad (\text{this assumes independence}) \\ &= \sum_{i \in [N]} \sum_{j \in [n_i]} \log L(y_{ij} | m_i) \\ &= \sum_{i \in [N]} \sum_{j \in [n_i]} \log f_{\text{err}}(y_{ij})(m_i(t_{ij})) \\ &= \sum_{\substack{i \in [N] \\ j \in [n_i]}} \log f_{\text{err}}(y_{ij})(m_i(t_{ij})). \end{aligned}$$

The independence assumption mentioned here is quite strong since we are talking about a time series of measured tumor size, which under the assumption that the tumor either grows or shrinks is necessarily autocorrelated. This makes such measurement generally dependent.

⁵Mixed effects models can be described by this formulation.

⁶Examples are additive or multiplicative errors under normal distributions.

2.3 Joint Models

Joint models incorporate a longitudinal process into a survival process [17]. This is done by augmenting the hazard function of the survival model using the longitudinal process.

Suppose we have a longitudinal model m , with individual realizations m_i for individuals $i \in [N]$ as described in 2.2 a and any distribution for the survival process. We call the hazard of this distribution h_0 the baseline hazard. Similarly there is a baseline PDF and survival function.

The joint model is the distribution after we augment the hazard function. One very common way [17] to augment this hazard is by multiplying an exponential containing a "link"⁷ $g(m_i(t))$ to the longitudinal process inside an exponential function. This gives us a different hazard formulation for all individuals $i \in [N]$ which depend on longitudinal models.

$$h_i(t) = h_0(t) \exp(\beta \cdot g(m_i(t)))$$

Previous implementations have joined these models by including the link function as part of an exponential term similar to the Cox proportional hazard model [7] and the benefits shown here 2.1. This work will focus on implementing the established formulation. It should be noted that other means of augmenting the hazard formula would be possible.

The hazard formulation could also be extended by incorporating other links to the same or different longitudinal processes or covariates for the individuals.

Likelihood

From these individual hazard formulations we can derive the formulations of the individual PDF f_i and survival S_i according to survival analysis 2.1. Furthermore, we can calculate the likelihood that an event will occur for censored or uncensored events according to 2.1.

Suppose $\theta = (\theta_s, \theta_b, \theta_l)$ fully describes a parametric joint model. In particular, θ_s describes the baseline survival distribution, θ_b the link, and θ_l describes the longitudinal model $\{m_i\}_{i \in [N]}$ with its error model. And we have survival $Y_S = ((t_i, \delta_i))_{i \in [N]}$ and longitudinal $Y_L = \{(t_{ij}, y_{ij})_{i \in [N], j \in [n_i]}$ measurements. Notice that we are working with two time variables t_i for survival times and t_{ij} for longitudinal measurements. Given the parameters the log likelihood of the

⁷In literature often denoted by f but here I choose g to separate its name from the probability density function.

joint model can be calculated as follows:

$$\begin{aligned}
l(Y_S, Y_L | \theta) &= \log(L(Y_S | \theta) L(Y_L | \theta_l)) \\
&= \log \left(\prod_{i \in [N]} L((t_i, \delta_i) | \theta) \prod_{i \in [N], j \in [n_i]} L((t_{ij}, y_{ij}) | \theta_l) \right) \\
&= \sum_{i \in [N]} \log L((t_i, \delta_i) | \theta) + \sum_{i \in [N], j \in [n_i]} \log L((t_{ij}, y_{ij}) | \theta_l) \\
&= \sum_{i \in [N]} \log(f_i(t_i)^{\delta_i} \cdot S_i(t_i)^{1-\delta_i}) + \sum_{i \in [N], j \in [n_i]} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \log(f_i(t_i)^{\delta_i} \cdot S_i(t_i)^{1-\delta_i}) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \delta_i \cdot \log(f_i(t_i)) + (1 - \delta_i) \log(S_i(t_i)) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \delta_i \cdot \log(f_i(t_i)) - (1 - \delta_i) H_i(t_i) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))).
\end{aligned}$$

The log-likelihood of joint models is given as the sum of the survival and longitudinal measurements, where the survival log-likelihood is described by the censoring process 2.1 and the longitudinal log-likelihood has an error model as described in 2.2.

These calculations are the basis for the implementation of joint models. The log-likelihood calculations are needed inside the Turing framework for Bayesian sampling algorithms [10].

Link functions To fully understand such joint models, we need to examine the link functions g considered in the literature. Kerioui et al. [17] mention the identity link id , the derivative $\frac{d}{dt}$ and the time to growth (TTG) as a suitable link function for their work. The TTG is domain specific to their longitudinal model.

$$\begin{aligned}
g_{id}(m_i(t)) &= m_i(t) \\
g_{\text{derivative}}(m_i(t)) &= \frac{d}{dt} m_i(t) \\
g_{\text{TTG}}(m_i(t)) &= \text{TTG}(m_i) \quad (\text{constant function in time})
\end{aligned}$$

Here we note that the link function g is not a function on $\mathbb{R}$ but the function space containing the longitudinal models m_i . And in case of Kerioui et al. [17] it might be that we do not know g_{TTG} itself, only the image $g_{\text{TTG}}(m_i(t))$ for a particular longitudinal model formulation⁸.

⁸They specify g_{TTG} for their longitudinal model SLD

Chapter 3

Methods and Design

I made interesting findings concerning the implementation of joint models in Julia. Please, take a look at the workflows in the Appendix A. They sum up the tools I have developed myself and all the packages I have found useful in the Julia/Turing ecosystem. The following sections summarize my thoughts and results, but generally do not contain code or data.

3.1 Towards implementation of Joint Models

To start with this project I implemented a survival model A.2 and longitudinal model A.3 separately.

For the survival process, the main difficulty for the implementation in Turing is the censoring. At first, I used a trick to access the survival probability by wrapping the survival function (ccdf) S_i inside a Bernoulli distribution:

```
Bernoulli(ccdf(Weibull( $\alpha$ ,  $\theta$ ), time_obs[i]))
```

This allows access to the survival function to calculate the log likelihood for a censored event.

Toward the end of my project, I found the censored functionality provided by the Distributions.jl package [3]. The reason I did not consider this earlier is that for a major part of the project duration I added the log likelihood manually.

Implementing the longitudinal process was straightforward, since Turing allows us to call standard Julia code inside the domain-specific probabilistic programming language. We can write our own model function and then use it inside the probabilistic programming language Turing. The main challenge here was to understand the mixed effects and how to built them. In the section on the mathematical formulation of the longitudinal process 2.2 I abstract this complexity by giving each individual a different model realization.

In the Appendix A.3 there are two examples of longitudinal mixed effects models with different parameter setups. In the end, all such formulations end with a vector of parameters per individual. Implementing these mixed-effects priors is straightforward, after understanding their mathematical description.

First Joint Model To implementing joint models we calculate both the likelihood for longitudinal and survival data. For the survival data, we cannot work with parametric¹ survival distributions anymore. The Turing.jl framework allows us access to the likelihood calculation, specifically the log likelihood via the function `Turing.@addlogprob!()`. We can manually specify the log likelihood according to our calculations in 2.3 for the survival data. To calculate the integral in the cumulative hazard, I used a Gauss-Konrad numerical integration procedure provided by `QuadGK.jl` [16]. This package was originally part of the Julia language, but has been made a standalone package.

After having first implemented a joint model, two functional limitations of that early implementation arose. First, some advanced diagnostic tools did not work. One example is the leave-one-out cross-validation (LOO-CV) through Pareto smoothed importance sampling² [24] provided by `ParetoSmooth.jl` [5]. This is because we lose the connection between the observation and likelihood contribution for the survival observations inside the framework when adding the log likelihood manually.

Another limitation was that predictions based on the posterior samples cannot be generated inside the Turing framework using the manual log-likelihood contributions. There is a `predict()` function in the Turing framework that uses the model formulation and posterior samples [10] to generate posterior predictions. For each posterior sample, it instantiates the model with these values and generates sample observations. But this uses the `rand()` functionality implemented by distributions inside the `Distributions.jl` package [3]. If we add the log probability ourselves we do not provide this functionality.

Custom Distribution The root of these two problems is that now the Turing framework does not have a distribution associated with the survival data. The solution I found is to implement a distribution type for joint models. More generally I developed a distribution type for distributions based on the hazard formulation. See Chapter 4 for a complete documentation.

I created an abstract type `HazardBasedDistribution`. This type is capable of computing the survival and probability density function based on a given hazard formulation³, which allows us to implement the log-likelihood contributions for censored data. In Turing, we can use structs of this type with the Turing

¹Distributions with names such as Weibull, Normal etc.

²State of the art diagnostic for Bayesian model development.

³According to their mathematical formulations presented in chapter 2.

tilde syntax to denote likelihood calculations A.2. Which allows us to calculate the LOO-CV statistic mentioned above.

To make predictions using the Turing framework I implemented the `rand()` function for my type `HazardBasedDistribution`. I applied the inverse transform sampling, also described in section 4.1. This allows us to generate event data based on the posterior of a Bayesian sampling which allows us to generate diagnostics such as posterior predictive checks.

Design Given the development of the project the design for a user interface for implementing joint models changed quite drastically to guarantee the functionalities described above. I came up with ideas such as using Turing sub-models [10] or extending the `@formula` macro from StatsModels.jl [8].

In the end, the necessity of creating my own distribution gave me a foundation to develop a cohesive interface. The distribution hides the calculations and presents the models relevant choices A.4 such as the baseline model or the link function. These need to be interchangeable. Therefore, a flexible interface for them is crucial.

The design goals for the interface I set up⁴ are:

- Intuitive and direct access to prior distributions.
- Simple but explicit formulation of the likelihood for survival and longitudinal processes.
- New distributions based on the hazard formulation should be simple to implement. In particular, this allows one to implement joint models.
- Lightweight structure around the Turing framework to minimize maintenance work and the complexity⁵.

3.2 Case Study: Simulated Data

After implementing a toy example the model I used as the primary use case and target for development was the joint model described by Kerioui et al. [17]. My implementation can be found in A.4. It also explains the details of this joint model. I will not go into details in this section and just showcase some parts.

This specific joint model is interesting because its longitudinal model $SLD(t, \Psi)$ is quite complex and non-linear. We have $\Psi = (BSLD, g, d, \varphi) \in \mathbb{R}^4$, which

⁴These can be altered with feedback from users who implemented joint models using `HazardBasedDistribution`.

⁵The framework first transforms the `@model` function. It is not a function but rather an description of a Bayesian model in Turing [10].

3. METHODS AND DESIGN

describes the mixed effects for an individual.

$$\text{SLD}(t, \Psi) = \begin{cases} \text{BSLD} \exp(gt) & \text{if } t < t_x \\ \text{BSLD} \exp(gt_x) \cdot [\varphi \exp(-d(t - t_x)) + (1 - \varphi) \exp(g(t - t_x))] & \text{if } t \geq t_x \end{cases}$$

Every individual has a different Ψ vector that is built by population and random effects. More details about the priors can be found in the Appendix A.4 or in the paper.

Accompanying the paper [17] Kerioui et al. provided a simulated data set with 655 total measurements of 100 individuals. This data was generated according to their model with given parameters.

Parameter	Value
μ_{BSLD}	60
μ_d	0.0055
μ_g	0.0015
μ_φ	0.2
σ	0.18
κ	1
λ	1450
β	0.01

To test my implementation of their model I am comparing my posterior samples to the parameters they used to generate the data. Here I am sampling 200 samples after 100 burn in samples using the NUTS [12] algorithm provided by the Turing framework. The statistics about the posterior samples:

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat
Symbol	Float64	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	61.0555	4.6970	0.5009	89.9910	89.3489	1.0195
μ_d	0.0058	0.0011	0.0001	116.4744	160.1451	1.0248
μ_g	0.0018	0.0002	0.0001	6.5780	104.2091	1.1427
μ_φ	0.1587	0.0581	0.0438	1.8077	28.8366	1.6211
σ	0.1763	0.0064	0.0006	121.8497	151.1699	1.0078
κ	1.0866	0.0750	0.0152	23.0509	55.6639	1.0130
λ	1359.2824	265.8978	22.9817	151.1063	178.6288	1.0031
β	0.0102	0.0010	0.0001	253.7573	152.8143	1.0045

Table 3.1: This statistics is generated by the Turing framework after sampling posterior samples. The rows describe the parameters for which posterior samples were generated. The mean and std describe the mean value and standard deviations of the posterior samples. The mcse estimates the Monte carlo mean squared error. The ess estimates the effective sample size and rhat estimates the $\hat{R}$ statistics commonly used in Bayesian statistics.

Most of the mean values of the posterior samples are relatively close to the parameters used to generate the data. An exception is the μ_ϕ parameters, which are off by quite a bit. We can see in the statistics about the posterior that its rhat value is high 1.6 and the mean squared error, which is high in relation to the value of μ_ϕ . To get better posterior distributions we can increase the burn in duration and the sample size of the posterior. This will increase the sampling duration which is a trade-off.

The output of the sampling call is not just statistics of the posterior but we also have access to the actual samples that were generated. Using them we can generate diagnostic plots. See 3.1 as an example of an individual diagnostic plot.

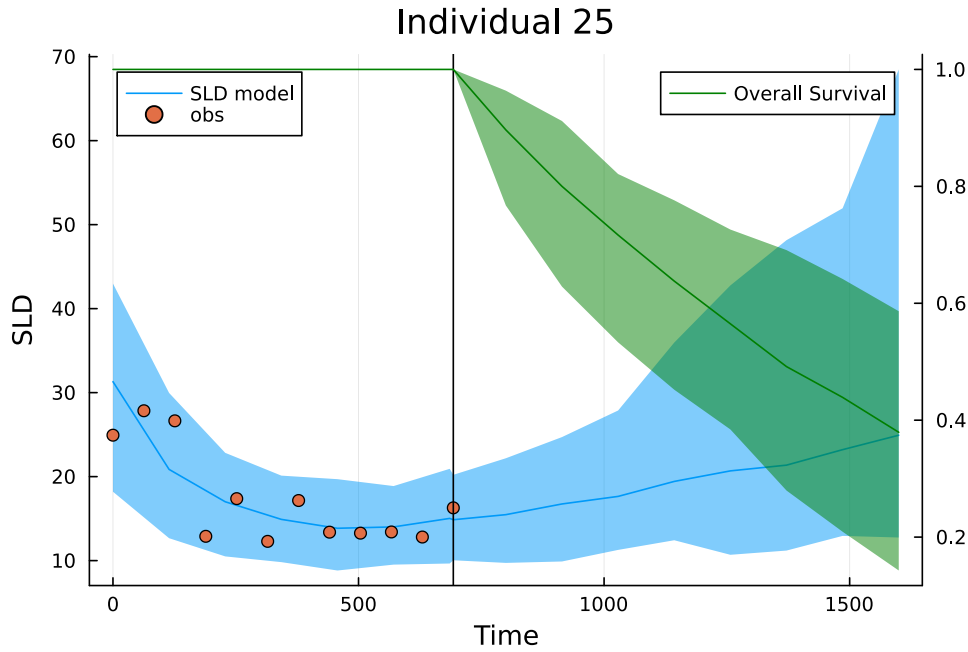


Figure 3.1: Non-parametric posterior predictive plot I made for joint models. Individual 25 is the first censored individual in data set. In blue is the posterior predictive 95% quantile of the longitudinal model and the line is the median while the orange dots are the measured values. In green is the posterior predictive 95% quantile of the survival via Kaplan-Meier estimators. See A.2 for a detailed explanation.

3.3 Diagnostics

While creating the workflows, I have developed many diagnostic plots. There are two categories of plots. One are the posterior predictive checks, where we use the Turing framework to generate sample measurements based on the

posterior distribution⁶. The other plots analyze specific values such as the cumulative hazard for Cox-Snell and Martingale residuals for survival models and the individual predictions for longitudinal models.

DiagnosticHelper / Dirac To save values such as the cumulative hazard $H(t)$ or individual predictions (IPRED) during the Turing sampling call I developed a helper distribution `DiagnosticHelper`. It takes a value α and when sampled it returns this value α . In other words the `DiagnosticHelper` represents a distribution where every sample is α .

Using this, we can save the cumulative hazard or IPRED into the output of the Bayesian sampling of the posterior. See the application in A.2 A.3.

When thinking about the name to give to this distribution, I realized that it is isomorphic to the Dirac distribution [9]. The distribution $\text{Dirac}(\alpha)$ has cumulative probability density function

$$F_{\alpha}(x) = \begin{cases} 1 & x \geq \alpha \\ 0 & x < \alpha \end{cases}.$$

Sampling a value from this distribution always gives α .

The package `Distributions.jl` [3] already has the Dirac distribution implemented. Therefore my `DiagnosticHelper` is replaced by this Dirac implementation. The functionality to save generated values is an important tool to analyze our models. I applied this method to generate Cox-Snell, Martingale, IPRED vs OBs and IRES vs covariates plots.

Posterior Predictive Check After sampling the posterior distribution of a model we can use the posterior distribution to sample observation values. The posterior predictive check allows us to detect if our model is not capable to describe certain observations and it allows one to compare different model implementations.

In the case of a mixed-effects longitudinal model, we can specify the time points at which we want to generate samples⁷. In the output of the Turing function `predict`, we predict a value for each individual and each time point, for each sample in the posterior. See the Appendix for more details A.3.

For each individual this allows us to calculate the 95% quantile at the specified time points as well as the mean, median or mode. We can then plot these values alongside the observed data. See 3.2 for an example plot.

⁶The posterior distribution was previously sampled with a Bayesian sampling algorithm from the Turing framework.

⁷Note that the measurement error is included here.

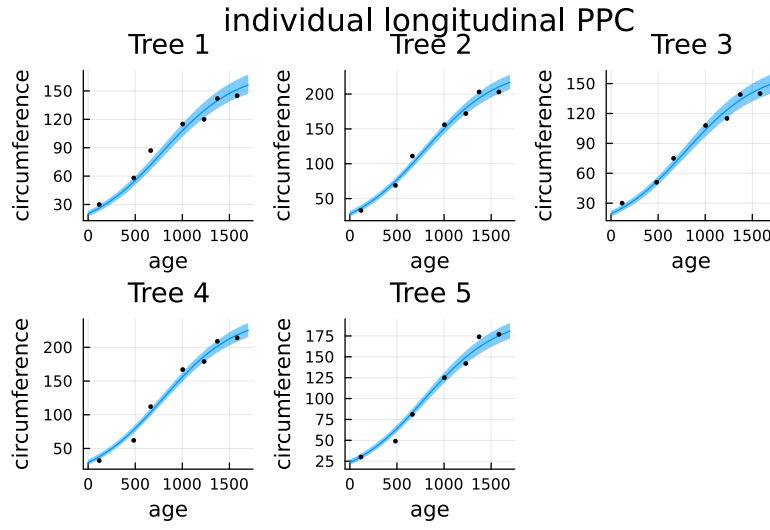


Figure 3.2: Posterior predictive check for a longitudinal model, which describes the tree circumference over time. See A.2. The black dots are the observations while the blue band represents the 95% quantile of the posterior predictive for each tree.

For survival models the predicted event times according to the posterior are not in the preferred format. We want to view the survival function and not the events themselves. For this we can use a non-parametric estimator of the survival function, such as the Kaplan-Meier estimator provided by the `Survival.jl` package [2]. See 3.3 for an example plot and the Appendix A.2 for the implementations.

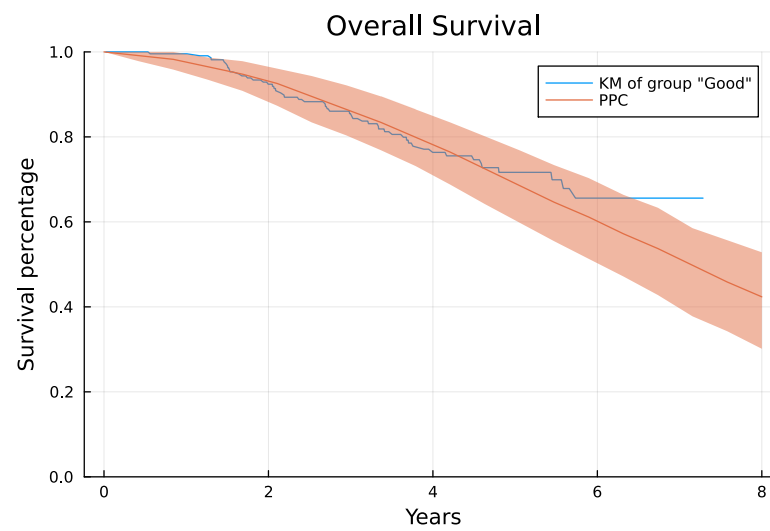


Figure 3.3: Posterior predictive check for a survival model A.2 First we calculate a Kaplan-Meier estimator for the observation data. Then we calculate a Kaplan-Meier estimator for each of the posterior predictions. From these we calculate the 95% quantile interval at different time points.

3.4 Comparison to Stan

To evaluate my implementation we are consider the joint model specified in the paper by Kerioui et al. [17] and the simulated data provided with this paper. The other implementations we are consider are the original code attached with the paper and the package `jmpost` [11] that is currently under development.

First, I compare the posterior statistics produced by the original code and my implementation as an argument for a degree of confidence about the correctness of implementation. Afterwards, I will compare the interface and functionality to the `jmpost` package in development.

Numerical Comparison I will compare the sampling output of my implementation vs the implementation done by Kerioui et al. [17]. For the sampler we use the NUTS [12] sampler with 200 warm-up samples and a target acceptance rate of 0.9. In the posterior, we sample 400 samples. We use the same prior distributions in both implementations and supply initial values for the sampling algorithm according to their specification in [17]. The table 3.2 are the statistics about the posterior sampled with the R/Stan code provided along with the paper.

	mean	se_mean	sd	n_eff	Rhat
mu_1	62.26464	0.61117	4.48100	54	1.03979
mu_2	0.00487	0.00024	0.00197	65	1.06537
mu_3	0.00181	0.00007	0.00029	20	1.15857
mu_4	0.20024	0.01110	0.08510	59	1.07103
Omega[1]	0.73227	0.00650	0.05196	64	1.07858
Omega[2]	0.56993	0.05826	0.43380	55	1.07908
Omega[3]	0.83742	0.01418	0.11937	71	1.04076
Omega[4]	1.54100	0.02664	0.35360	176	1.00189
sigma	0.17672	0.00026	0.00602	545	1.00097
lambda	1505.44167	13.41165	339.75674	642	0.99912
beta	0.01032	0.00005	0.00134	652	1.00179

Table 3.2: This statistics is generated by the Stan framework [22]. Similar to the Turing output the rows represents model parameters and the columns certain statistics.

And the statistics about the posterior generated with Julia code similar to the one found in the Appendix A.4 in table 3.3.

3. METHODS AND DESIGN

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat
Symbol	Float64	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	61.2455	4.4382	0.6618	44.3845	116.2119	1.0013
μ_d	0.0055	0.0014	0.0001	150.5143	199.8976	1.0042
μ_g	0.0018	0.0002	0.0000	104.7299	59.5683	1.0681
μ_ϕ	0.1839	0.0605	0.0057	108.7283	232.6829	1.0040
ω_{BSLD}	0.7378	0.0573	0.0021	701.9017	285.8221	1.0137
ω_d	0.3100	0.1598	0.0555	5.9030	34.0874	1.1270
ω_g	0.8736	0.1071	0.0096	123.3917	324.7381	1.0035
ω_ϕ	1.4668	0.2676	0.0308	74.4418	125.7581	1.0259
σ	0.1772	0.0064	0.0004	282.0313	249.8288	0.9989
λ	1512.4779	322.9993	16.3085	510.1417	199.3771	1.0031
β	0.0104	0.0013	0.0001	455.9621	329.9539	1.0052

Table 3.3: This statistics is generated by the Turing framework [10]. Similar to the above the rows represents model parameters and the columns certain statistics.

Considering the mean of the posterior diagnostics we can see that both sampling calls produce posteriors that resemble the parameters used to generate the data. The main exception is ω_d for which both the Julia implementation ω_d and the Stan/R implementation $\Omega[2]$ both estimate a value that is too low. This appears to be due to the randomness of the specific data set. In both cases, the value of $\hat{R}$ is high, indicating this discrepancy. This is however only clearly readable in the statistics from the Julia implementation, since its the main rhat value that is far off from one. In the Stan/R implementation, multiple rhat values are far off from one.

Design Finally, I want to take a few paragraphs to compare the interface and joint model implementation of my implementation with the one provided by `jmpost` [11].

In the Julia/Turing implementation A.4, using the data type I developed, we specify a joint model inside a function with the `@model` macro annotation. In this function we describe the prior distributions and likelihood calculations. To specify the longitudinal model, we need to write a function that represents this model. For the joint model, we need to specify the link and baseline hazard.

Afterwards, we instantiate the model with observations and call a sampling algorithm which tries to generate samples of the posterior distribution.

The `jmpost` package wraps the Stan/C++ code and R bindings to give us access to joint models. We can specify the prior setup with the function call `JointModel()`, which takes longitudinal and survival distributions that are provided by the `jmpost` package.

This is currently limited to the joint model described by Kerioui in [17] and a random slope model with a Weibull or an exponential survival distribution. After creating a model, we load the data and call a sampling function that calculates the posterior samples.

The benefits of the jmpost implementation are the clear direction of what models are possible and how to load the data. The workflow I present to implement joint models is more flexible what substituting longitudinal or joint models is concerned, as well as sampling algorithm setup. My workflow allows general implementations of joint models in addition to the one featured by Kerioui et al. [17].

Chapter 4

Implementation of Joint Models

This chapter describes the technical implementation of joint models in Julia according to the `Distributions.jl` [3] package. This `HazardBasedDistribution` type allows to formulate joint models and then use them inside the Turing framework for probabilistic programming and Bayesian sampling [10].

4.1 Type: `HazardBasedDistribution`

The `HazardBasedDistribution` type implements a survival distribution based on a given formulation of the hazard function h . In general, we assume that the cumulative hazard H does not have an analytical form. Distributions used for survival analysis [6] such as the Weibull or Log-logistic distributions have a analytical form. Joint models [17] can have a cumulative hazard which does not have an analytical representation. Therefore we will use numeric integration to calculate the cumulative hazard.

The section on mathematical formulations 2.1 describes how the hazard $h(t)$ and the cumulative hazard $H(t) = \int_0^t h(u) du$ functions can formulate the survival function $S(t) = \exp(-H(t))$ and the probability density function $f = h(t) \cdot S(t) = h(t) \cdot \exp(-H(t))$.

To represent distributions based on the hazard formulation, I implement a type¹ that can integrate the hazard function to get the cumulative hazard². We do this with a Gauss-Konrad procedure provided by the `QuadGK.jl` package[16].

```
1 abstract type HazardBasedDistribution <:
  ↳ ContinuousUnivariateDistribution end
```

¹That subtypes `ContinuousUnivariateDistribution` of the `Distributions.jl` [3] package of Juila.

²If you know the analytical formulation of the cumulative hazard of your struct you can also overload the hazard function with that formulation.

4. IMPLEMENTATION OF JOINT MODELS

```
2
3 # represents  $h(t)$ 
4 function hazard(dist::HazardBasedDistribution, t::Real) end
5
6 # calculates  $H(t) = \int_0^t h(u) du$  numerically with a Gauss-Konrad
  ↪ procedure
7 function cumulative_hazard(dist::HazardBasedDistribution, t::Real)
8     # integrate from 0.0001 to  $t$  (some hazards are not defined at 0)
9     /h = IntegralProblem(hazard, 1e-4, t, dist)
10    H = solve(/h, QuadGKJL())[1]
11    return H
12 end
```

We can then formulate the survival `ccdf()` and `pdf()` based on the formulas we derived in subsection 2.1. The `logpdf()` is also implemented which is called by the likelihood calculation inside Turing.jl framework [10].

```
1 function ccdf(dist::HazardBasedDistribution, t::Real)
2     exp(- cumulative_hazard(t, dist))
3 end
4
5 function pdf(dist::HazardBasedDistribution, t::Real)
6     hazard(dist, t) * ccdf(dist, t)
7 end
8
9 function logpdf(dist::HazardBasedDistribution, t::Real)
10    log(pdf(dist, t))
11 end
```

To use the `predict()` functionality of the Turing framework we also need to be able to generate samples using the `rand()` functionality on our distribution [10]. To generate a random sample, we use the ODE solver of the `DifferentialEquations.jl` package [18] to calculate the cumulative hazard $H(t)$ over an interval.

Then we apply inverse transform sampling to create a random sample. With the calculated cumulative hazard, we can formulate the CDF $F(t)$. We generate a random sample $x_0 \in U(0,1)^3$ and search for the time t_0 such that $F(t_0) = x_0$. For this, let us formulate $g(t) = F(t) - x_0$ and search the root t_0 such that $g(t_0) = 0$ which then gives us the sample we search for. In code this is achieved by a root finding algorithm provided by the `Roots.jl` package [25].

³This is done via a uniform random number generator present in Julia [4].

```

1 function rand(rng::AbstractRNG, dist::HazardBasedDistribution)
2     num_end = 1_000_000.
3     # calculate the ode solution of  $H(t) = \int_0^t h(u) du$ 
4     H(nLogS, p, t) = hazard(t, dist)
5     prob = ODEProblem(H, 0.0, (1e-4, num_end))
6     sol = solve(prob, Tsit5(); p=dist)
7     # calculate cdf  $F(t)$ , via survival  $S(t)$ 
8     S(t) = exp(-sol(t))
9     F(t) = 1 - S(t)
10    # inverse sampling: find  $t_0$  such that  $F(t_0) = x_0$  where  $x_0 \sim U(0,1)$ 
11    # via the root of  $g(t) := F(t) - x_0$ 
12    x0 = rand()
13    g(t) = F(t) - x0
14    return find_zero(g, (1e-4, num_end))
15 end

```

4.2 Usage

To showcase the functionality, we will first implement a log-logistic distribution and then a joint model.

LogLogistic

```

1 struct LogLogistic <: HazardBasedDistribution
2     λ
3     α
4 end
5
6 function hazard(t::Real, dist::LogLogistic)
7     α, λ = dist.α, dist.λ
8     return λ*α*(λ*t)^(α-1) / (1 + (λ*t))
9 end

```

This is all we have to define to be able to calculate the pdf, logpdf, survival, cumulative hazard function and to generate random samples. In particular we can use this distribution in the Turing framework to model our data. In the plot 4.1 I plotted the numerical pdf as well as a normalized histogram of the generated samples.

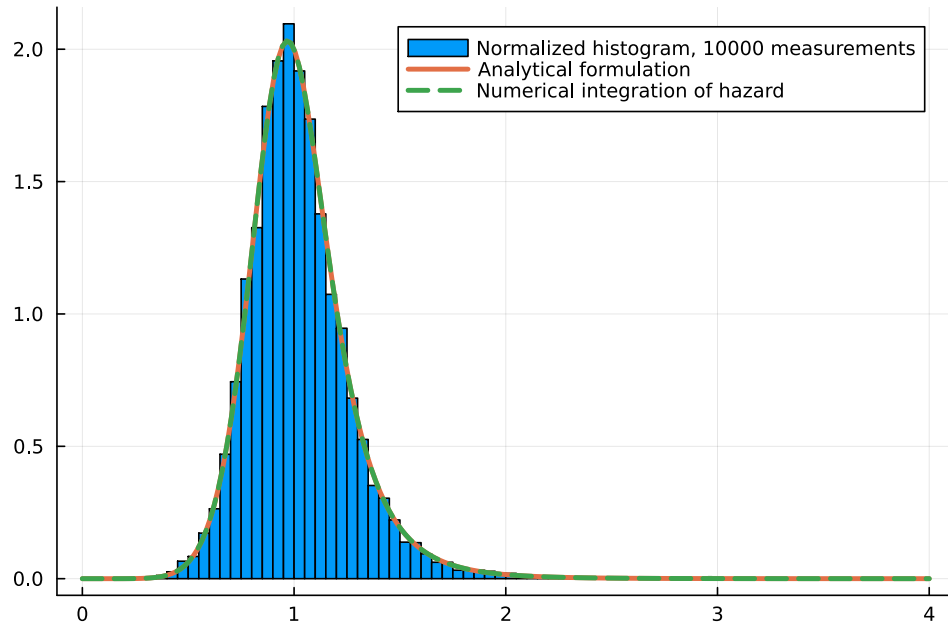


Figure 4.1: Example of $\text{LogLogistic}(1,8)$ implemented with the code above. On the plot you see the analytical pdf in orange as well as the numerical pdf in a dashed green line calculated by the `HazardBasedDistribution` type `numerical_pdf(x) = pdf(LogLogistic(1,8), x)`. The histogram is normalized to a discrete probability density and represents 10'000 samples generated with `rand(LogLogistic(1,8), 10_000)`. See A.2 for the code.

Joint Model

To implement a joint model 2.3 we need to save the baseline survival distribution, the link coefficient β , and the link $t \mapsto g(m(t))$ described as a function of time. To make use of the functionality provided by `HazardBasedDistribution`, we implement a hazard function

$$h(t) = h_0(t) \cdot \exp(\beta \cdot g(m(t)))$$

of a joint model. In code:

```

1 struct SimpleJointModel <: HazardBasedDistribution
2   base_survival::ContinuousUnivariateDistribution
3   β::Real
4   link::Function # function of time
5 end
6
7 h₀(d::SimpleJointModel, t::Real) = hazard(d.base_survival, t)
8
```

```

9 function hazard(d::SimpleJointModel, t::Real)
10     return h0(d, t) * exp(d.β * d.link(t))
11 end

```

I want to highlight the simplicity of implementation and the mapping to the mathematical formulation. The abstract type `HazardBasedDistribution` allows one to implement a joint Model in two steps. First we define a `struct` that saves the data. This `struct` corresponds to the $\theta = (\theta_s, \theta_b, \theta_l)$ mentioned in the likelihood calculation 2.3 of the joint model. Then we implement the `hazard` according to our mathematical joint model formulation.

To make use of all the distributions in the `Distributions.jl` package, we add the `hazard` function. This allows us to make use of them as a baseline survival distribution for a joint model. We can do this using $h(t) = \frac{f(t)}{S(t)}$ since the pdf f and cdf S are defined⁴ for all the distributions in `Distributions.jl` [3].

```

1 hazard(dist::ContinuousUnivariateDistribution, t::Real) = pdf(dist, t)
  ↪ / cdf(dist, t)

```

The `SimpleJointModel` distribution can now be used with a baseline distributions from the `Distributions.jl` package. Since it inherits from the `HazardBasedDistribution` type we can evaluate the pdf and the survival/cdf functions according to their mathematical definitions. And it has the functionality to generate samples of this joint distribution. This is what we need [10] to use this distribution in the Turing.jl probabilistic programming language.

Another point is the recursive definition of `SimpleJointModel`. For the baseline distribution, we can also use another `SimpleJointModel` with its own baseline distribution. This will extend the final hazard formulation to incorporate another link.

$$h(t) = \overline{h_0(t)} \cdot \exp(\overline{\beta} \cdot \overline{g(m(t))}) = \underbrace{h_0(t) \cdot \exp(\beta \cdot g(m(t)))}_{\overline{h_0} \text{ is a joint model}} \cdot \exp(\overline{\beta} \cdot \overline{g(m(t))})$$

This allows⁵ to incorporate different links to different or the same longitudinal process.

⁴Note this calculation will raise an error if the cdf is 0.

⁵A different formulation of the `struct` and `hazard` can be helpful for clarity if you want to design a joint model with multiple links.

Chapter 5

Conclusion

The Julia ecosystem prides itself on interoperability between packages of different domains that work together seamlessly. Working with Julia was simple and straightforward, and the Turing framework with its probabilistic programming language opens interesting doors.

For example in my work I have used a Gauss-Konrad numerical integrator, a differential equation solver and a root finding algorithm, provided by packages in the Julia ecosystem, to implement a distribution for joint models that can be used inside the Turing probabilistic programming framework for Bayesian sampling. The Turing documentation shows examples of implementations of Bayesian estimation of differential equations using `DifferentialEquations.jl` [18], as well as Bayesian neural networks using the main neural network package in Julia called `Flux.jl` [14].

The resulting workflows that you can find in the Appendix A shows how simple the implementation and exploration of joint models can be. This relies on the extendability and interoperability of the Julia ecosystem. I conclude that the Julia programming language is well suited for exploring joint models and developing scientific tools.

One fact not to forget is that the Julia language and the associated packages are rather new. On one hand this means less maturity of the available tools compared to languages such as SAS or R, which first appeared in 1972 and 1993 respectively. On the other hand, this gives the opportunity to do things differently. I hope this work was a convincing demonstration that doing things differently has enormous potential.

Appendix A

Appendix

The main product I handed in at Roche is a documentation website I created to summarize the tools I have developed and discovered in the Julia ecosystem. It showcases workflows for implementing survival, mixed effects longitudinal and joint models.

I have included the PDF¹ versions of the documentation. I have used Quarto [1] to generate this documentation. Its intended format was a website and not a PDF, thus some of the formatting is not ideal.

A.1 Further Exploration

Here is a list of possible further explorations:

- Explore link functions on data: Link functions represent the main idea of joint models. The workflows show how easy it is to explore the link functions.
- Compositional Samplers: Turing offers a compositional Gibbs sampler which could split the search space for longitudinal and survival parameters.
- Submodels: The longitudinal model would be suitable for a submodel. This might lead to an interface for switching longitudinal models.
- AD Mode: HMC algorithms use automatic differentiation. I have focused on using ReverseDiff.jl [15], but there are others supported, such as Zygote.jl [13].

A.2 Survival Workflow

This workflow is included in the edition of the thesis with appendix.

¹Here meaning the Dataformat not probability density function.

A.3 Longitudinal Workflow

This workflow is included in the edition of the thesis with appendix.

A.4 Joint Molder Workflow

This workflow is included in the edition of the thesis with appendix.

A.5 Julia in Latex

Julia allows you to use many different unicode characters. This helps with readability and allows you to reduce errors when implementing mathematical formulas and ideas.

The problem when representing Julia code in Latex is that most mono-fonts never implemented these extended unicode characters. The solution I used in this document (and the quarto renderings) is to provide my own monofont via a folder in my Latex project. I used JuliaMono for this and downloaded the font from <https://juliamono.netlify.app/>.

When you have the JuliaMono folder in you project you can use it with the fontenc package.

```
\usepackage{fontspec}
\setmonofont{JuliaMono-Regular.ttf}[
  Path = ./JuliaMono/,
  Contextuals = Alternate,
  CharacterVariant=1,
  Scale = MatchLowercase,
  BoldFont = JuliaMono-Bold.ttf,
  %Numbers = OldStyle
]
```

For this to work you need to use LuaLaTeX or XeLaTeX to render the PDF.

Bibliography

- [1] J.J. Allaire, Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. Quarto, January 2022.
- [2] Alex Arslan. Survival.jl: Survival analysis in julia. <https://github.com/JuliaStats/Survival.jl>, 2018.
- [3] Mathieu Besançon, Theodore Papamarkou, David Anthoff, Alex Arslan, Simon Byrne, Dahua Lin, and John Pearson. Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem. *Journal of Statistical Software*, 98(16):1–30, 2021.
- [4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, September 2017.
- [5] Carlos Parada <cdp49@cam.ac.uk>. Paretosmooth.jl, 6 2021.
- [6] David Collett. *Modelling Survival Data in Medical Research*. Chapman and Hall/CRC, May 2015.
- [7] D. R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society. Series B (Methodological)*, 34(2):187–220, 1972.
- [8] Milan Bouchet-Valat Dave Kleinschmidt. StatsModels.jl: Specifying, fitting, and evaluating statistical models in julia. <https://github.com/JuliaStats/StatsModels.jl>, 2017.
- [9] P. A. M. Dirac. *The Principles of Quantum Mechanics*. Clarendon Press, 1930.
- [10] H Ge, K Xu, and Z Ghahramani. Turing: A language for flexible probabilistic inference. 2018.

- [11] Craig Gower-Page, Georgios Kazantzidis, Daniel Sabanes Bove, Xiaoyan Fang, Francois Mercier, and Isaac Gravestock. *jmpost: Joint Models for Predicting Overall Survival Trajectories*, 2023. R package version 0.0.0.9000.
- [12] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo, 2011.
- [13] Michael Innes. Don’t unroll adjoint: Differentiating ssa-form programs. *CoRR*, abs/1810.07951, 2018.
- [14] Michael Innes, Elliot Saba, Keno Fischer, Dhairya Gandhi, Marco Concetto Rudilosso, Neethu Mariya Joy, Tejan Karmali, Avik Pal, and Viral Shah. Fashionable modelling with flux. *CoRR*, abs/1811.01457, 2018.
- [15] Mohamed Tarek Jarrett Revels. ReverseDiff.jl: Reverse mode automatic differentiation for julia. <https://github.com/JuliaDiff/ReverseDiff.jl>, 2019.
- [16] Steven G. Johnson. QuadGK.jl: Gauss–Kronrod integration in Julia. <https://github.com/JuliaMath/QuadGK.jl>, 2013.
- [17] Marion Keroui, Francois Mercier, Julie Bertrand, Coralie Tardivon, René Bruno, Jérémie Guedj, and Solène Desmée. Bayesian inference using hamiltonian monte-carlo algorithm for nonlinear joint modeling in the context of cancer immunotherapy. *Statistics in Medicine*, 39(30):4853–4868, 2020.
- [18] Christopher Rackauckas and Qing Nie. Differentialequations.jl – a performant and feature-rich ecosystem for solving differential equations in julia. *The Journal of Open Research Software*, 5(1), 2017. Exported from <https://app.dimensions.ai> on 2019/05/05.
- [19] Dimitris Rizopoulos. JM: An R package for the joint modelling of longitudinal and time-to-event data. *Journal of Statistical Software*, 35(9):1–33, 2010.
- [20] Dimitris Rizopoulos. The R package JMBayes for fitting joint models for longitudinal and time-to-event data using mcmc. *Journal of Statistical Software*, 72(7):1–45, 2016.
- [21] Elisabeth Roesch, Joe G. Greener, Adam L. MacLean, Huda Nassar, Christopher Rackauckas, Timothy E. Holy, and Michael P. H. Stumpf. Author correction: Julia for biologists. *Nature Methods*, 20(5):771–771, April 2023.

- [22] Stan Development Team. Stan modeling language. <https://github.com/JuliaStats/StatsModels.jl>, 2012.
- [23] Stan Development Team. CmdStanR: the r interface to cmdstan. <https://mc-stan.org>, 2020.
- [24] Aki Vehtari, Andrew Gelman, and Jonah Gabry. Practical bayesian model evaluation using leave-one-out cross-validation and waic. *Statistics and Computing*, 27(5):1413–1432, Sep 2017.
- [25] John Verzani. Roots.jl: Root finding functions for julia. <https://github.com/JuliaMath/Roots.jl>, 2020.